API Documentation

Base URL	Auth Method	Database	Version
http://localhost:5000/api	JWT via Authorization: Bearer	MongoDB (Mongoose)	1.0.0

1. Authentication — /api/auth

No JWT token required for these endpoints.

POST `/api/auth/register` — Public

Register a new user. Passwords are hashed with bcrypt (10 rounds). Returns a signed JWT valid for 7 days.

Body: { "name": "string*", "email": "string*", "password": "string*", "calorie_goal": "number", "dietary_prefs": "string" }

Response: 201: { "token": "JWT", "user": { id, name, email } } | 400: Email already exists

POST `/api/auth/login` — Public

Authenticate with email and password. Returns a JWT token to use in the Authorization header.

Body: { "email": "string*", "password": "string*" }

Response: 200: { "token": "JWT", "user": { id, name, email } } | 400: User not found / Wrong password

2. User — /api/user

GET `/api/user/profile` — JWT Required

Retrieve the profile of the currently authenticated user. User object decoded from the JWT token.

Response: 200: { "message": "User profile fetched", "user": { decoded JWT payload } } | 401: No/Invalid token

3. Meals — /api/meals

Full CRUD for user-specific meals. Each meal is owned by the authenticated user.

POST `/api/meals` — JWT Required

Create a new meal. Automatically linked to the authenticated user's ID.

Body: { "name": "string*", "description": "string", "ingredients": ["string"], "calories": "number" }

Response: 201: { "message": "Meal created", "meal": { _id, name, description, ingredients, calories, user } }

GET `/api/meals` — JWT Required

Retrieve all meals belonging to the authenticated user.

Response: 200: { "meals": [{ _id, name, description, ingredients, calories, user }] }

PUT `/api/meals/:id` — JWT Required

Update any field of an existing meal by its MongoDB ID.

Body: { "name"?, "description"?, "ingredients"?: ["string"], "calories"? }

Response: 200: { "message": "Meal updated", "meal": { updated } } | 500: error detail

DELETE `/api/meals/:id` — JWT Required

Permanently delete a meal by its MongoDB ID.

Response: 200: { "message": "Meal deleted" } | 500: error detail

4. Recipes — /api/recipes

Nutrition Auto-Calculation: When a recipe is created or updated with ingredients, the server automatically sums calories, protein, carbs, and fat across all ingredients and stores both totalNutrition and perServing breakdowns. GET endpoints are public; POST/PUT/DELETE require JWT. PUT/DELETE additionally enforce ownership.

POST `/api/recipes` — JWT Required

Create a new recipe. Nutrition totals and per-serving values are calculated server-side.

Body: { "title": "string", "cuisine": "string", "servings": "number", "ingredients": [{ name, calories, protein, carbs, fat }] }

Response: 201: { "message": "Recipe created", "recipe": { ..., "totalNutrition": {...}, "perServing": {...} } }

GET `/api/recipes` — Public

Retrieve all recipes. Supports optional query params: search (title regex), cuisine (exact), maxCalories (perServing.calories).

Response: 200: { "recipes": [array of recipe objects] }

GET `/api/recipes/:id` — Public

Retrieve a single recipe by its MongoDB ID.

Response: 200: { "recipe": { full recipe object } } | 404: Recipe not found

PUT `/api/recipes/:id` — JWT + Owner

Update a recipe. Ownership verified. Nutrition recalculated if ingredients are provided.

Body: { "title"?, "cuisine"?, "servings"?, "ingredients"?: [...] }

Response: 200: { "message": "Recipe updated", "recipe": { updated } } | 403: Not authorized | 404: Not found

DELETE `/api/recipes/:id` — JWT + Owner

Permanently delete a recipe. Only the original creator can delete it.

Response: 200: { "message": "Recipe deleted" } | 403: Not authorized | 404: Not found

5. Meal Plan — /api/mealplan

Weekly meal planning. One plan per user with 7 days (monday–sunday), each holding recipe references. All endpoints require JWT.

POST `/api/mealplan` — JWT Required

Create an empty weekly meal plan for the authenticated user. All 7 days start as empty arrays.

Response: 201: { "message": "Meal plan created", "plan": { _id, user, week: { monday:[], ..., sunday:[] } } }

GET `/api/mealplan` — JWT Required

Retrieve the authenticated user's meal plan with all recipe references populated.

Response: 200: { "plan": { _id, user, week: { monday: [{ recipe objects }], ... } } }

PUT `/api/mealplan/:day` — JWT Required

Append a recipe to a specific day. Valid days: monday–sunday.

Body: { "recipeId": "MongoDB ObjectId string (required)" }

Response: 200: { "message": "Recipe added to monday", "plan": {...} } | 400: Invalid day | 404: Plan not found

DELETE `/api/mealplan/:day` — JWT Required

Clear all recipes from a specific day by setting that day to empty.

Response: 200: { "message": "monday cleared", "plan": {...} } | 404: Plan not found

ERROR References

Code	Status	Cause	Description
200	**OK**	Success	Request processed successfully
201	**Created**	Resource created	POST succeeded; new document returned
400	**Bad Request**	Validation error	Missing required field, wrong password, duplicate email
401	**Unauthorized**	Auth failed	No token, expired token, or malformed token in header
403	**Forbidden**	Access denied	Token valid but user does not own the resource
404	**Not Found**	Resource missing	ObjectId does not exist in the database
413	**Payload Too Large**	Body limit exceeded	Request body exceeds 15 MB limit (e.g., large image)
500	**Server Error**	Internal error	Unhandled exception — check server logs for stack trace